

0–k BFS: Bounded Integer Weights Shortest Path Problem

Stage 1 Report – Algorithms Course, C++ Implementation

Artemov Makar, Mark Shkut *Skoltech, Algorithms 2026*

May 21, 2026

1 Scope and deliverables

We build and benchmark a C++ implementation of the 0–k BFS family of shortest-path algorithms and compare it against a Dijkstra baseline, scaling to the largest vertex count a commodity workstation can handle – our target was V approaching 10^9 . At that scale explicit edge storage is infeasible (a CSR with $E = 4V$ and 32-bit ids/weights costs ≈ 20 GB at $V = 10^9$), so the project distinguishes two graph backends: **GridGraph** (implicit 4-neighbour grid + per-vertex byte, up to $V \approx 10^9$) and **GraphCSR** (classical CSR, up to $V \approx 10^8$). The grid backend never materializes edges; **for_each_neighbour(u)** runs in $O(1)$. The dominant memory cost is the distance array, halved from 8 GB (`uint64_t`) to 4 GB (`uint32_t`) because $k \cdot V < 2^{32}$ holds for every parameter we use.

Deliverables: (i) four dataset generators (grid, ER, layered DAG, chain with chords); (ii) visualization (per-graph distance field + benchmark plots); (iii) pipeline runner with JSONL output; (iv) Dijkstra baseline; (v) 0–1 BFS and 0–k BFS (with $k+1$ circular FIFO queues). Every algorithm’s distance checksum was verified against Dijkstra on identical inputs; all 19 distinct graph instances from the 45-run parameter sweep cross-validate (zero mismatches).

2 Algorithms

(a) **dijkstra_pq (baseline)**. Standard Dijkstra with `std::priority_queue` and lazy decrease-key (push duplicates, skip stale pops). $O((V + E) \log V)$.

(b) **bfs_01**. `std::deque<Vertex>`: weight 0 $\rightarrow$ `push_front`, weight 1 $\rightarrow$ `push_back`. $O(V + E)$.

(c) **bfs_0k**. $k+1$ circular FIFO queues indexed by $\text{dist}[v] \bmod (k+1)$; the current processing distance `cur` walks forward, vertices re-relaxed land in another bucket, stale slots are skipped on pop. $O(kV + E)$, memory $O(k + E)$ for the queues plus $O(V)$ for distances. All three are templated over a graph adapter, so the same code runs on both backends.

3 Datasets and visualization

Four generators with seeded `std::mt19937_64` RNG: **grid** (rows, cols, k – the headline case for $V \rightarrow 10^9$), **er** ($G(n, p)$, weights $\in \{0..k\}$), **layered** (L layers of width W , fanout F), **chain** (path of n vertices + c random chords – adversarial). For the grid, $w(u, v) = (\text{terrain}[u] + \text{terrain}[v]) \bmod (k+1)$ produces structured weight landscapes that show up clearly in the heatmap (Fig. 1, left). Two visual layers come from JSON emitted by `zkbfs_dump`: per-graph distance fields (grid heatmap or node-link with colour = distance) and benchmark plots aggregated from the JSONL sweep.

4 Results

Headline numbers (grid, single thread).

V	k	Dijkstra	0–k BFS	0–1 BFS	best speed-up
10^6	1	0.180 s	0.039 s	0.070 s	4.6×
10^6	4	0.222 s	0.054 s	—	4.1×
10^7	1	1.98 s	0.493 s	0.355 s	5.6×
10^7	4	2.61 s	0.623 s	—	4.2×
10^8	1	60.3 s	28.4 s	5.44 s	11.1×
10^8	4	84.9 s	48.8 s	—	1.7×
$4.84 \cdot 10^8$	4	432.9 s	451.4 s	—	0.96× (mem-bound)
$9 \cdot 10^8$	1	619.9 s	—	121.2 s	5.12×

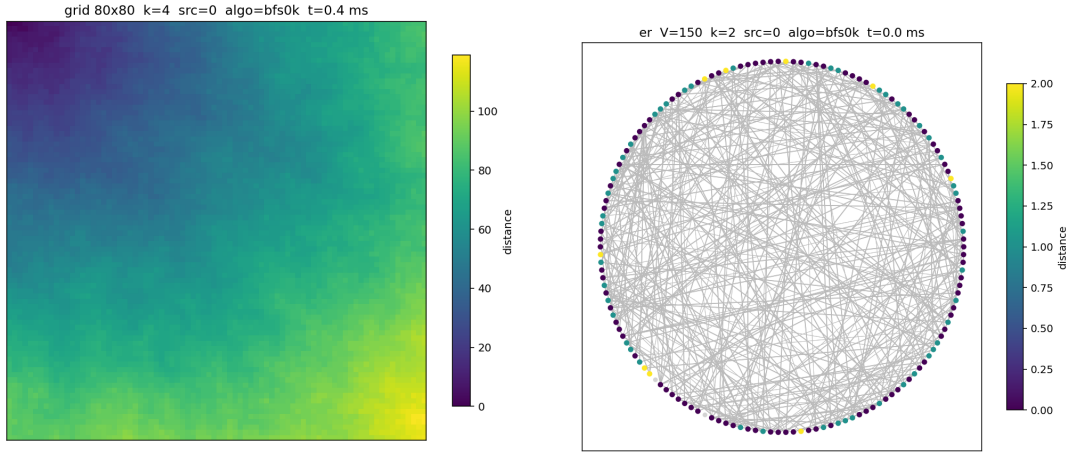


Figure 1: **Left:** 80×80 grid, $k = 4$, source at the top-left; the heatmap shows the distance field shaped by random terrain. **Right:** Erdős-Rényi graph, $V = 150$, $p = 0.03$, $k = 2$; edges in grey, vertex colour encodes distance from the source.

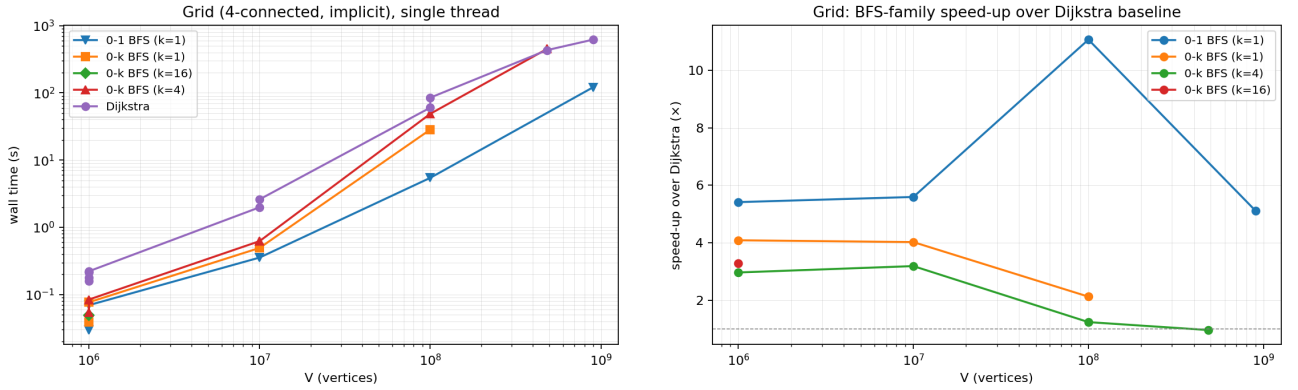


Figure 2: Implicit 4-connected grid, single thread. **Left:** wall time vs V (log-log). **Right:** BFS-family speed-up over the Dijkstra baseline; the dashed line marks parity. The 0-1 BFS via `std::deque` peaks at $\sim 11\times$ at $V = 10^8$ before memory pressure pulls it back.

$V \approx 10^9$. The largest run used a $30\,000 \times 30\,000$ implicit grid ($V = 9 \cdot 10^8$, $E = 3.6 \cdot 10^9$). Single thread, commodity RAM: Dijkstra 619.9s, 0-1 BFS **121.2s** – speed-up **5.12 $\times$** . Both produce the same distance-sum checksum 1703472107894, so correctness holds at the billion-vertex scale. Working set during the 0-1 BFS run was ≈ 8.5 GB: 3.6 GB for the `uint32_t` distance array, 0.9 GB for the terrain, ≈ 4 GB peak for the `std::deque` of pending vertices.

5 Limitations and observations

Crossover with k . As k grows the 0- k BFS speed-up shrinks: the $V = 10^6$ grid shows $4.6\times$ at $k = 1$, $4.1\times$ at $k = 4$, $3.3\times$ at $k = 16$. Asymptotically the advantage disappears at $k = \omega(\log V)$.

Memory-bandwidth ceiling. At $V \gtrsim 5 \cdot 10^8$ both algorithms become memory-bound: the queues alone are multi-gigabyte structures (peak push count $\approx 6 \cdot 10^8$), so the algorithmic advantage of 0- k BFS over a cache-friendly Dijkstra essentially vanishes (the $V = 4.84 \cdot 10^8$ row is $0.96\times$). The 0-1 BFS via a single `std::deque` stays competitive at $V = 9 \cdot 10^8$ because a deque is chunked and noticeably more cache-friendly than $k+1$ separate queues.

Implementation cost of `std::queue`. At $V = 10^8$, $k = 1$ the single-deque 0-1 BFS runs in 5.4s vs. the two-queue 0- k BFS at 28.4s. Same algorithmic complexity – the gap is pure allocator/cache overhead of maintaining $k+1$ FIFOs. A chunked ring-buffer FIFO is the planned Week 2 fix.

Lazy decrease-key. Our Dijkstra reinserts entries instead of supporting `decrease_key`; an indexed d -ary heap is queued for Week 2.

6 Next steps (Week 2)

(1) Indexed 4-ary heap Dijkstra; rerun the headline grid sweep. (2) Chunked ring-buffer FIFO in `bfs_0k` to close the gap against the deque-based 0–1 BFS. (3) Real road-network loader (DIMACS USA, weights quantised to $k = 8$). (4) Stretch goal: a Δ -stepping variant for context (parallel friendly).